

Research Design and Pilot Study Report

Alla Krylova 196722
Oleksandr Nychyporchuk 196659
Kyril Pahkevich 196687
Pavel Khmialeuski 197055

1 Research project

1.1 Title

The impact of service-mesh solutions on the efficiency of microservice applications

1.2 Supervisor

Krzysztof Gierłowski (KT)

1.3 Goals and short description

Currently, a significant portion of applications deployed in cloud environments utilize microservices architecture. This type of architecture requires decomposing application functionality into component elements, implemented by individual microservices, and then deploying them in an environment that allows the microservices to communicate efficiently and reliably. This task is most often accomplished by orchestration platforms such as Kubernetes. Maintaining the separation and security of applications sharing the same deployment platform is also a key aspect of this type of deployment. Service mesh solutions are often used for this purpose, as they can automatically create secure communication environments for specific applications.

The aim of this project is to analyze and experimentally test the impact of using service mesh solutions on the efficiency (and particularly performance) of an application composed of multiple microservices.

2 Research design

2.1 Research goal

The goal of this research is to empirically evaluate and compare the performance overhead introduced by different service mesh implementations (specifically Istio, Linkerd, Consul, and Istio Ambient) in a custom microservices application running on Kubernetes, compared to a baseline deployment without any service mesh. The custom application includes heterogeneous services: an AI agent service (running on a separate virtual machine for true isolation), media processing (video, digital filters), and mathematical computation modules (differential equations, integration, permutations, Fibonacci). The research aims to quantify the trade-offs between security features (mTLS) and performance metrics (latency, throughput, CPU/memory consumption, CPU clock usage) across these implementations.

2.2 Research gap

The systematic literature review identified 12 core articles on service mesh performance. Key findings from the SLR indicate:

1. Service meshes introduce measurable latency and resource overhead, primarily due to sidecar proxies and mTLS [1], [2].
2. Linkerd generally exhibits lower overhead than Istio under high load [3], but Istio remains the standard for complex traffic management.

3. Newer eBPF-based architectures (e.g. Istio Ambient) show promise in reducing overhead [1], [4].
4. Most studies focus on either single service mesh implementations or compare a limited set of meshes using standard benchmarks. However, there is a lack of controlled, reproducible experiments that compare multiple service meshes (including sidecar-less and eBPF-based variants) against a common baseline using a custom, heterogeneous microservice application that stresses different resource types (CPU-intensive math, I/O-oriented media, and off-cluster AI). Furthermore, few studies systematically vary security feature activation (mTLS on/off) to quantify the security-performance trade-off across meshes.

Our research will fill this gap by conducting a controlled experiment that measures the performance impact of adding different service meshes (Istio, Linkerd, Consul, Istio Ambient) to a purpose-built microservice application (designed to isolate throughput, CPU clock usage, RAM usage, and latency), with and without security features enabled, and compares them against a no-mesh baseline.

2.3 Research questions

- RQ1) How do different service mesh implementations (Istio, Linkerd, Consul, Istio Ambient) compare in terms of average latency and throughput relative to a baseline deployment without any service mesh, when running a heterogeneous microservice application (including an off-cluster AI service)?
- RQ2) What is the CPU and memory consumption overhead of each service mesh (including sidecar proxies and control planes) under increasing request loads, and how do they compare across different service types (math, media, off-cluster AI)?
- RQ3) What is the additional performance overhead specifically attributable to mTLS when enabled versus disabled within each service mesh, and how does this overhead differ among the four meshes?

2.4 Research hypotheses

RQ	Hypothesis (H)	Null Hypothesis (H_0)
RQ1	H_1 : Under high load conditions, the implementation of an Istio service mesh introduces a statistically significant increase in latency compared to the baseline application without a service mesh.	H_{01} : Under high load conditions, there is no statistically significant difference in latency between the Istio service mesh implementation and the baseline application.
RQ2	H_2 : Enabling mTLS within the Consul service mesh introduces a statistically significant performance overhead compared to the identical Consul setup with mTLS disabled.	H_{02} : Enabling mTLS within the Consul service mesh does not introduce a statistically significant performance overhead compared to the identical Consul setup with mTLS disabled.
RQ3	H_3 : Under identical load conditions, the Linkerd service mesh keeps statistically significantly higher throughput than the standard Istio service mesh.	H_{03} : Under identical load conditions, the Linkerd service mesh does not keep statistically significantly higher throughput than the standard Istio service mesh.
RQ4	H_3 : Under identical load conditions, the Istio Ambient mesh consumes statistically significantly less CPU than the standard Istio sidecar implementation.	H_{03} : Under identical load conditions, the Istio Ambient mesh does not consume statistically significantly less CPU than the standard Istio sidecar implementation.

All hypotheses are falsifiable and will be tested with statistical significance ($\alpha = 0.05$).

2.5 Research subjects and sample

In the context of this performance experiment, the subjects are the experimental configurations of the custom microservice application under test. Each configuration is a combination of:

- Service mesh presence: None (baseline) vs. Istio vs. Linkerd vs. Consul vs. Istio Ambient
- mTLS setting (if mesh): Disabled vs. Enabled
- Service type: AI Service (off-cluster), Media (Video + Filters), Mathematical (Differential Equations, Integration, Permutations, Fibonacci)

Every application configuration will be tested on 3 load levels:

- Low - 25 req/s
- Medium - 50 req/s
- High 100 req/s

Sample: We will use a full factorial design: Baseline ((1 configuration, no mTLS) + 4 meshes x 2 (mTLS on/off)) x 3 load levels = 3 + 24 = 27 distinct configurations. Each configuration will be tested 10 times across 3 service types → total experimental runs = 27 x 10 x 3 = 810 runs.

Study population: The custom application must be deployed as a set of containerized microservices (except the AI service, which runs on a separate VM), each exposing a REST endpoint. The infrastructure must be a dedicated Kubernetes cluster plus an isolated VM for the AI service.

The application is designed to stress different system resources:

- **AI Service** (off-cluster): heavy compute, requires load caps, strict isolation - communicates with the cluster via a dedicated ingress.
- **Media Services:** video compression, frame splitting, matrix filters (memory and I/O).
- **Mathematical Modules:** CPU-intensive (differential equations, integration, permutations $O(n!)$, Fibonacci for clock time).

Evaluating the entire application comprehensively is beyond the scope of this initial pilot study due to its overall scale and complexity. Therefore, to ensure precise measurement and tightly controlled variables, our experiments will focus exclusively on isolated, singular aforementioned modules of the application.

Sampling method: Non-probabilistic - we select the services that represent typical microservice communication patterns (request-response, chain calls, parallel fan-out, and hybrid cluster-external calls).

2.6 Operationalization - variables

Variable type	Name	Operational definition / Measurement
Independent	Service mesh type	Categorical: baseline (no mesh), Istio, Linkerd, Consul, Istio Ambient
Independent	mTLS setting	Categorical: disabled vs. enabled (only applied to mesh conditions)
Independent	Request load	Continuous: offered load in requests per second (req/s), varied at 25, 50, 100 req/s
Independent	Request Payload	Categorical: specific data inputs sent with the request, representing different traffic profiles (e.g., '1KB JSON', '5MB Image file', 'Standard text prompt')

Independent	Service Module	Categorical: the specific isolated application microservice group processing the request (e.g., 'Math computation', 'Media processing', 'Off-cluster AI proxy')
Dependent	Average latency	Mean request-response time (ms) per service endpoint over a 60-second steady-state window, reported as p50, p95, p99 percentiles
Dependent	Throughput	Maximum sustained requests per second achieved per service before error rate exceeds 1%
Dependent	CPU usage	Average CPU millicores consumed by mesh sidecars + application containers (cluster side), separated per service
Dependent	Memory usage	Average resident memory (MiB) consumed (cluster side), separated per service
Dependent	CPU clock usage	Observed time of request processing on math nodes under Fibonacci load
Confounding	Network conditions	Controlled by running all tests in the same isolated cluster and VM network
Confounding	Application version	Fixed version of the custom application across all runs
Confounding	Kubernetes version	Fixed version across all runs
Hidden	Garbage collection cycles	Minimized by running each test for a sufficient duration (warm-up + measurement) and averaging over repetitions
Hidden	Node heterogeneity	Use identical instances for all worker nodes; AI VM kept identical across runs

2.7 Research methods

This research will employ a controlled real-life experiment conducted in a cloud-based Kubernetes environment, with an additional isolated VM hosting the AI service. The experiment is reproducible and quantitative. No human subjects are involved; only system performance metrics are collected.

Primary method: Laboratory experiment with repeated measures (each configuration is tested multiple times) and blocking on service type to account for heterogeneity.

2.8 Research tools

Experimental setup:

- **Cloud provider:** Local Kubernetes cluster + separate VM for AI service.
- **Orchestration:** Kubernetes (version 1.28+)
- **Service meshes:**
 - Istio (latest stable, e.g., 1.22) with default sidecar injection
 - Linkerd (latest stable, e.g., 2.15) with default sidecar injection
 - Consul (latest stable, e.g., 1.18) with sidecar injection
 - Istio Ambient (latest stable, eBPF-based, sidecar-less data plane)
- **Custom microservice application:** Built by the team, consisting of:

- **AI Service:** runs on a separate VM - receives requests via Kubernetes ingress, performs inference/test data generation.
- **Media Service:** containerized service for video splitting and digital filters.
- **Math Services:** C# implementations of differential equations, integration, permutations, Fibonacci (with configurable n), running inside the cluster.
- **Load generator:** A separate virtual machine (different from the AI VM) running a custom Python script that sends HTTP requests to the ingress of the Kubernetes cluster. The script controls the request rate (25, 50, 100 req/s) and targets each service endpoint (including the AI service via its external endpoint). The load VM is isolated from both the cluster and the AI VM to avoid interference.
- **Metrics types:**
 - Mesh-specific telemetry (Istio/Ambient, Linkerd, Consul)
 - Kubernetes metrics-server (for container CPU/memory)
 - Custom metrics for CPU clock usage
- **Metrics collection, storage, and visualization mechanisms:**
 - Data Generation & Exposure: Service mesh proxies intercept all network traffic to calculate telemetry (latency, throughput) and expose it via local /metrics web endpoints. Infrastructure data (CPU millicores, memory) is exposed via Kubernetes cAdvisor. Custom application metrics (inner measurements of microservices) are written to local .prom files by the Load Generator and exposed using the Prometheus Node Exporter's textfile collector.
 - Data Collection (Scraping): Prometheus acts as the central aggregator. Utilizing Kubernetes Service Discovery, Prometheus automatically detects all active mesh proxies and infrastructure nodes. It executes a "pull" (scrape) of these /metrics endpoints at a fixed interval (e.g., every 15 seconds).
 - Data Storage (Prometheus): Prometheus functions as the sole data repository for measurements. It stores all scraped infrastructure, mesh, and custom metric data as distinct time-series rows in its highly optimized, local Time-Series Database (TSDB) on the hard drive.
 - Data Visualization and Aggregation (Grafana): Grafana is utilized strictly as the visualization layer and does not store the telemetry data. When a dashboard is accessed, Grafana dynamically executes PromQL queries against the Prometheus API, aggregating the raw metric data across multiple pods on the fly to render visual graphs.

Experiment design:

Testing procedure for the metrics generation, acquisition and aggregation consists of the following wteps:

- RQ1) Deploy application in the needed configuration (baseline, Istio, Linkerd, Consul, Istio Ambient).
- RQ2) Execute test runs 10 times for a given service type per configuration for every load level.
- RQ3) Aggregate and save data on a hard disk
- RQ4) Each test run consists of:
 - 40s warm-up
 - 120s steady-state measurement
 - Cooldown

Experiment iterations:

- RQ1) Execute the testing procedure for the baseline application.
- RQ2) Execute the testing procedure for Istio (mTLS-disabled) configuration.
- RQ3) Execute the testing procedure for Istio (mTLS-enabled) configuration.
- RQ4) Execute the testing procedure for Linkerd (mTLS-disabled) configuration.
- RQ5) Execute the testing procedure for Linkerd (mTLS-enabled) configuration.

- RQ6) Execute the testing procedure for Consul (mTLS-disabled) configuration.
- RQ7) Execute the testing procedure for Consul (mTLS-enabled) configuration.
- RQ8) Execute the testing procedure for Istio Ambient (mTLS-disabled) configuration.
- RQ9) Execute the testing procedure for Istio Ambient (mTLS-enabled) configuration.

2.9 Expected results

Quantitative:

- Latency distributions (p50, p95, p99) per service type for each configuration and load level.
- Throughput saturation curves.
- CPU and memory overhead percentages relative to baseline, broken down by service type and mesh.
- CPU clock usage behavior on math services (Permutations) under each mesh vs. baseline.

Qualitative:

- Insights into whether the overhead is constant or scales with load and service type.
- Recommendations for practitioners: which mesh to choose based on workload characteristics (CPU-intensive math, I/O media, off-cluster AI, or mixed).

2.10 Validity threats

Threat type	Description	Mitigation
Construct validity	Measured metrics may not fully represent “efficiency” (e.g., latency alone ignores user experience).	Use multiple metrics (latency, throughput, resource usage, clock scaling).
Internal validity	Uncontrolled variables (e.g., network jitter, node scheduling) affect results.	Run all tests on isolated, dedicated cluster and separate VMs; repeat each condition 10 times; randomize order.
External validity	Results may not generalize to other applications, cloud providers, or service meshes.	Use a custom application that covers diverse workload types (including off-cluster AI); test four representative meshes (Istio, Linkerd, Consul, Ambient); discuss limitations.
Conclusion validity	Random chance may produce false significance.	Use appropriate statistical tests, set $\alpha = 0.05$, and report effect sizes and confidence intervals.

2.11 Research plan

Phase	Tasks	Responsible	Duration
RQ1) Application development	Build and containerize AI (VM), Media, Math services; create Kubernetes deployment manifests; set up AI VM networking	All team members	5 days

RQ2) Environment setup	Provision Kubernetes cluster, install Istio, Linkerd, Consul, Istio Ambient; deploy custom app; validate baseline and AI VM connectivity	Oleksandr Nychyporchuk, Kiryl Pahkevich	4 days
RQ3) Load generator & monitoring	Develop custom load script on separate VM; configure Prometheus; create measurement scripts targeting each service (including AI VM)	Pavel Khmialeuski, Alla Krylova	2 days
RQ4) Pilot study	Run 1 repetition of a subset of configurations (baseline, Istio, Linkerd) for Math service to validate toolchain	Alla Krylova (execution), Oleksandr Nychyporchuk (verification)	1 day
RQ5) Full experiment	Execute 10 repetitions per configuration x 3 service types (750 runs), collect raw data	All team members (divided runs)	10-14 days
RQ6) Data analysis	Statistical tests, visualization, hypothesis testing	Oleksandr Nychyporchuk, Kiryl Pahkevich	4 days
RQ7) Reporting	Write research paper / report sections	Alla Krylova (draft), Oleksandr Nychyporchuk (final)	4 days

2.12 Publication goals

The results of this research are suitable for publication in:

- **Conference:** IEEE International Conference on Cloud Computing (CLOUD), IEEE International Conference on Edge Computing (EDGE), or International Conference on Software Engineering (ICSE) - SEIP track.
- **Journal:** Journal of Systems and Software, IEEE Transactions on Network and Service Management, or Cluster Computing.

The pilot study results alone may serve as a short paper or a technical report. The full experiment, if significant, can be submitted as a full conference paper.

3 Pilot study

3.1 Research subjects

For the pilot study, we used a subset of the experimental configurations to verify that all tools work and produce plausible data. The pilot focused on the Mathematical Modules (specifically the Permutations service $O(n!)$) as the most CPU-intensive. The pilot included 7 configurations (single repetition each) using only Istio and Linkerd (Consul and Ambient were not yet installed):

- RQ1) Baseline - low, medium, high load (25, 50, 100 req/s) - 3 runs
- RQ2) Istio (mTLS off) - low, medium, high load - 3 runs
- RQ3) Linkerd (mTLS off) - high load only (100 req/s) - 1 run

We omitted mTLS-on conditions and other service types (AI, Media) as well as Consul and Istio Ambient in the pilot due to time constraints. These will be included in the full experiment.

Qualification: The pilot used the same hardware and software versions as planned for the full experiment, except that the AI VM was not used (only cluster services).

3.2 Study execution

- **Executed by:** Oleksandr Nychporchuk
- **Verified by:** Kiryl Pashkevich
- **Date:** 02.05.2026
- **Environment:** Local Kubernetes cluster hosting the application infrastructure with a separated virtual machine for load generation
- **Tools:** Prometheus, Grafana
- **Procedure:**
 - RQ1) Deploy baseline (no service mesh) application.
 - RQ2) Execute the testing procedure described in chapter 2.8.
 - RQ3) Install Istio with default sidecar injection, disable mTLS, redeploy application.
 - RQ4) Repeat step 2.
 - RQ5) Install Linkerd with default sidecar injection, disable mTLS, redeploy application.
 - RQ6) Repeat testing porcedure with high-load test only (100 req/s).

3.3 Results

3.3.1 CPU Usage Comparison: Idle to Peak Load (100 req/s)

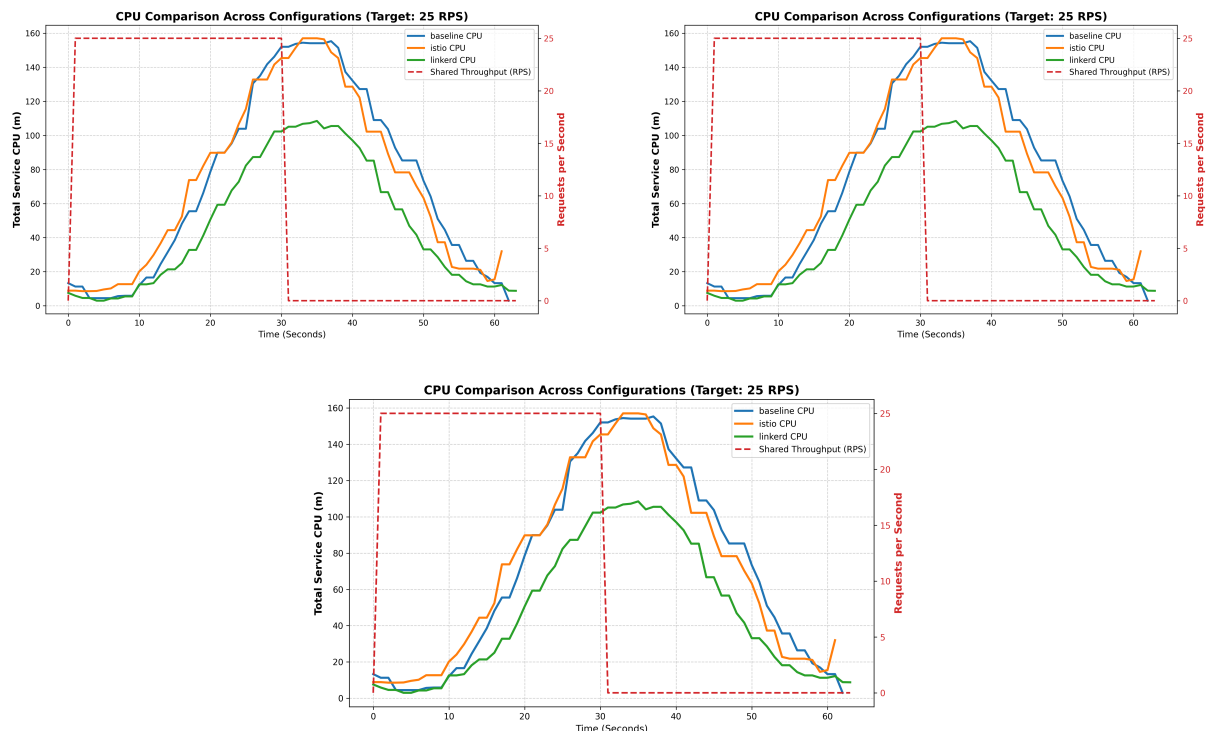


Figure 1: CPU usage — 25 req/s (top-left), 50 req/s (top-right), 100 req/s (bottom-center)

The architectural cost of the service mesh becomes evident when tracking CPU consumption from idle to peak load.

Baseline (No Mesh): The application alone consumes very little CPU at idle (10.8 m). Under peak load, it scales aggressively to 473.4 m, representing pure application processing without proxy overhead.

Linkerd: Starts with an extremely low idle CPU of 3.7 m—even lower than baseline. During the load test, CPU consumption rises smoothly, peaking at 437.0 m. This indicates that the Linkerd-proxy (built in Rust) adds relatively small overhead, with total CPU usage ending slightly lower than baseline.

Istio: Has a higher idle CPU baseline of 18.0 m due to the more resource-intensive Envoy proxy. Under peak load, CPU spikes more aggressively, peaking at 401.1 m. While the absolute peak is lower than both baseline and Linkerd, Istio consistently consumes more CPU during the ramp-up phase, showing a higher marginal cost per request at moderate loads.

Key CPU Insight: Linkerd is the most efficient overall, showing minimal overhead and a smooth scaling curve. Istio has a higher baseline cost but a lower absolute peak than baseline.

3.3.2 Memory Usage Comparison: Idle to Peak Load (100 req/s)

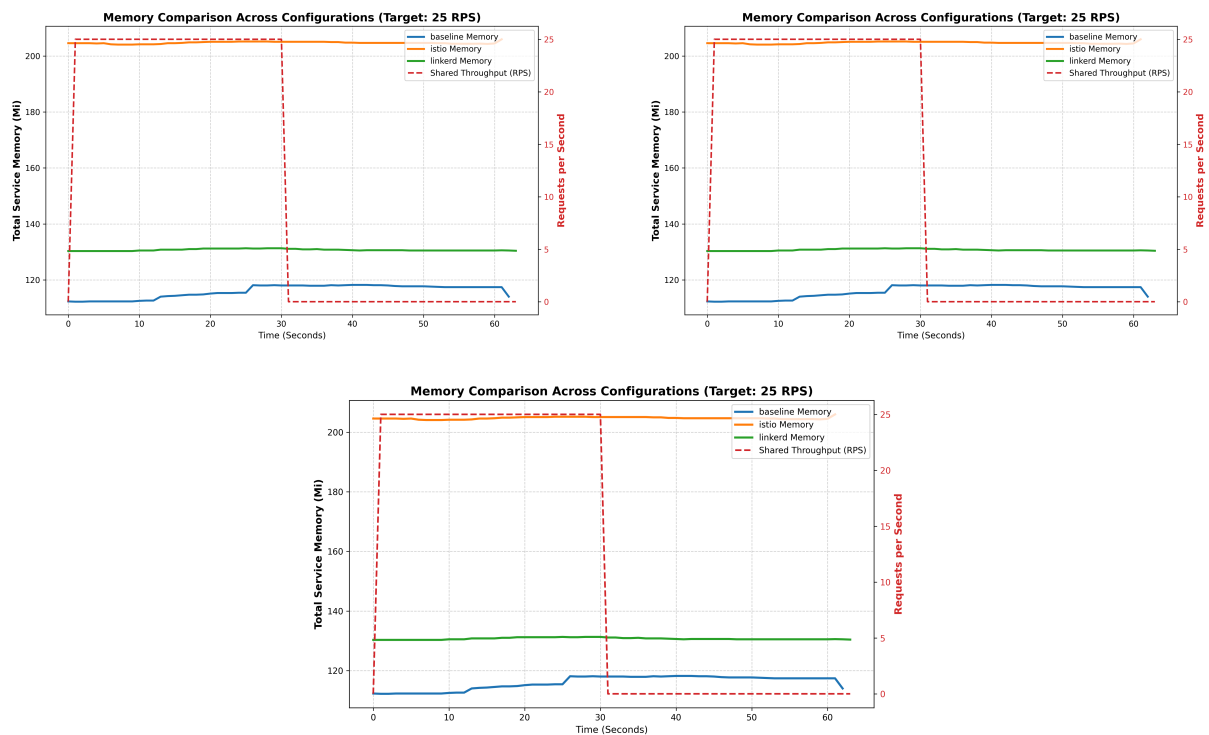


Figure 2: Memory usage — 25 req/s (top-left), 50 req/s (top-right), 100 req/s (bottom-center)

Memory footprint clearly differentiates the architectures, as proxies have a constant baseline memory cost regardless of traffic.

Baseline (No Mesh): Low memory baseline of 120.6 Mi. At peak load, memory usage grows modestly to 122.4 Mi, confirming that stateless applications have a small, stable memory footprint per request.

Linkerd: Idle memory is 132.0 Mi, which is 11.4 Mi higher than baseline due to the Linkerd-proxy sidecar. At peak load, memory only rises to 134.1 Mi (only +2.1 Mi from idle). This proves that

Linkerd has a very stable memory overhead, and additional traffic does not significantly increase memory usage.

Istio: Idle memory is 208.4 Mi, a massive 87.8 Mi higher than baseline, reflecting the larger Envoy proxy. At peak load, memory grows to 210.3 Mi (only +1.9 Mi from idle). Like Linkerd, memory overhead is constant, but the constant itself is 76 Mi higher than Linkerd's overhead.

Key Memory Insight: Linkerd introduces a small, constant memory overhead of 11.4 Mi. Istio introduces a large, constant memory overhead of 87.8 Mi. Neither mesh increases memory significantly under load; the cost is the proxy's static footprint.

3.4 Conclusions

CPU Overhead: The service mesh sidecar proxy introduces a measurable CPU overhead that scales with traffic. Linkerd demonstrated the lowest CPU impact, with idle consumption starting below baseline and peak load CPU remaining slightly under the no-mesh configuration. Istio consistently consumed more CPU during the entire ramp-up phase due to its heavier Envoy proxy, resulting in a higher marginal cost per request. Both meshes added overhead, but Linkerd's Rust-based proxy proved significantly more CPU-efficient.

Memory Overhead: The memory cost of a service mesh is predominantly a static footprint imposed by the sidecar proxy, not a dynamic cost that grows with traffic. Linkerd introduced a small, constant memory overhead, while Istio imposed a substantially larger constant overhead due to the Envoy proxy's higher baseline memory requirements. In both cases, memory usage remained nearly flat from idle to peak load, confirming that proxy memory consumption is a fixed infrastructure tax independent of request volume.

Efficiency Verdict: Linkerd offers the better resource efficiency profile overall, with near-negligible CPU overhead and a modest memory footprint. Istio provides comparable traffic handling but at a consistently higher resource cost, particularly in memory, where its idle overhead alone represents a significant fraction of total pod resources.

4 Conclusions

4.1 Design

Adjustments to Load Tiers: Because the 100 req/s peak load failed to stress the Baseline architecture, the subsequent main study must expand its load generation capabilities. The revised test matrix will introduce a "Stress" tier of 1000 req/s to accurately observe how the proxy architectures behave under compute saturation and network congestion.

Re-introducing Excluded Variables: The pilot successfully validated the core metrics pipeline and the sidecar injection methodology. With this foundational framework proven, the main study is structurally cleared to re-introduce the previously excluded variables. This will include deploying the Consul and Istio Ambient architectures, as well as executing the secondary test matrix to measure the specific computational overhead of enabling mutual TLS (mTLS) across all meshes.

4.2 Pilot study

Preliminary Validation of Hypothesis 1 (Baseline Overhead): The pilot data affirmatively supports the hypothesis that injecting any service mesh implementation creates a measurable latency and resource increase. The unmeshed Baseline consistently outperformed Istio in raw p95 latency and total memory footprint, confirming that the advanced routing and observability features of a service mesh incur a strict, non-zero architectural tax.

Linkerd vs. Istio Throughput (Hypothesis 3):

Preliminary findings strongly indicate that Linkerd provides a more efficient data plane than standard Istio, characterized by a significantly lighter memory footprint. At peak load (100 req/s), Linkerd's proxies consumed 36% less memory (134.1 Mi vs 210.3 Mi) than Istio's Envoy proxies, while both meshes sustained identical maximum throughput. Istio showed 8% lower CPU consumption (401.1 m vs 437.0 m), likely due to different proxy threading models. This suggests that for memory-constrained environments, Linkerd offers a distinct advantage, while CPU-constrained workloads may benefit from Istio's different scaling characteristics.

5 Literature

Bibliography

- [1] A. B. Barr, O. Lavi, Y. Naor, S. Rampal, and J. Tavori, "Technical Report: Performance Comparison of Service Mesh Frameworks: the mTLS Test Case," technical report arXiv:2411.02267, 2024.
- [2] M. Ganguli, S. Ranganath, S. Ravisundar, A. Layek, D. Ilangovan, and E. Verplanke, "Challenges and Opportunities in Performance Benchmarking of Service Mesh for the Edge," in *2021 IEEE International Conference on Edge Computing (EDGE)*, Chicago, IL, USA: IEEE, 2021, pp. 78–85. doi: 10.1109/EDGE53862.2021.00020.
- [3] K. Bosquez, X. Caiza, L. Nunez, and J. Monar, "Comparative Evaluation of Linkerd and Istio Service Meshes in a Microservices Architecture Application," in *2025 IEEE Colombian Caribbean Conference (C3)*, Santa Marta, Colombia: IEEE, 2025, pp. 1–6. doi: 10.1109/C366505.2025.11340184.
- [4] W. Yang, P. Chen, G. Yu, H. Zhang, and H. Zhang, "Network shortcut in data plane of service mesh with eBPF," *Journal of Network and Computer Applications*, vol. 222, p. 103805, 2024, doi: 10.1016/j.jnca.2023.103805.
- [5] M. Nagy, T. Lévai, F. Németh, A. Panda, G. Antichi, and G. Rétvári, "Elastic Scaling of Real-Time Communication Services," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 3393–3405, 2026, doi: 10.1109/TNSM.2026.3674598.
- [6] I. Dermentzis, G. Koukis, and V. Tsaoussidis, "Trade-Offs in Kubernetes Security and Energy Consumption," *Urban Science*, vol. 10, no. 2, p. 81, 2026, doi: 10.3390/urbansci10020081.
- [7] H.-Y. Huang, Y.-Y. Fanjiang, C.-H. Hung, H.-Y. Tsai, and B.-H. Lin, "Evaluation of a Smart Intercom Microservice System Based on the Cloud of Things," *Electronics*, vol. 12, no. 11, p. 2406, 2023, doi: 10.3390/electronics12112406.
- [8] S. Singh, C. H. Muntean, and S. Gupta, "Resilient microservices: an investigation into Istio effectiveness in Kubernetes," *Cluster Computing*, vol. 29, no. 1, p. 27, 2026, doi: 10.1007/s10586-025-05750-x.
- [9] L. Larsson, W. Tärneberg, C. Klein, E. Elmroth, and M. Kihl, "Impact of etcd Deployment on Kubernetes, Istio, and Application Performance," technical report arXiv:2004.00372, 2020.
- [10] D. A. Hahn, D. Davidson, and A. G. Bardas, "Security Issues and Challenges in Service Meshes – An Extended Study," technical report arXiv:2010.11079, 2020.
- [11] Q. Zeng, M. Kavousi, Y. Luo, L. Jin, and Y. Chen, "Full-stack vulnerability analysis of the cloud-native platform," *Computers & Security*, vol. 129, p. 103173, 2023, doi: 10.1016/j.cose.2023.103173.

- [12] S. Kurpad, S. Bt, S. Vijaykumar, S. Jain, and S. Kalambur, "Microarchitectural Analysis and Characterization of Performance Overheads in Service Meshes with Kubernetes," in *2023 3rd Asian Conference on Innovation in Technology (ASIANCON)*, Ravet IN, India: IEEE, 2023, pp. 1–6. doi: 10.1109/ASIANCON58793.2023.10270428.